PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Charles Santos de Souza Meira
Paulo Dimas da Silva Júnior

# Trabalho Prático 1: Unidade Lógica e Aritmética (ULA) de 8 bits em VHDL

Belo Horizonte

2026

Charles Santos de Souza Meira
Paulo Dimas da Silva Júnior

# Trabalho Prático 1: Unidade Lógica e Aritmética (ULA) de 8 bits em VHDL

Relatório do Trabalho Prático 1 apresentado à disciplina Sistemas Reconfiguráveis do curso de Engenharia de Computação da Pontifícia Universidade Católica de Minas Gerais, referente ao projeto, à simulação e à validação de uma unidade lógica e aritmética de 8 bits descrita em VHDL.

Pontifícia Universidade Católica de Minas Gerais

Professor: Francisco Garcia

Belo Horizonte
2026

# RESUMO

Este relatório descreve o projeto de uma unidade lógica e aritmética (ULA) de 8 bits, com 16 operações lógicas, aritméticas, de deslocamento e de rotação, descrita em linguagem VHDL exclusivamente com código concorrente e sintetizada no Quartus II 9.1sp2. O circuito é puramente combinacional e gera, além do resultado, cinco sinalizadores (negative, overflow, zero, half carry/borrow e carry/borrow). São apresentados a especificação, as decisões de projeto, o código comentado e os casos de teste, incluindo exemplos de soma fornecidos no enunciado. A validação foi feita por simulação funcional, comparando os valores obtidos com os valores esperados calculados a partir da especificação.

**Palavras-chave**: VHDL. ULA. Código concorrente. Lógica combinacional. Quartus II.

# LISTA DE ILUSTRAÇÕES


Figura 1 – Relatório de compilação (*Flow Summary*) no Quartus II 9.1sp2 . . . . . 16
Figura 2 – Simulação da operação PASS_B (op_sel = 0000) . . . . . 17
Figura 3 – Simulação da operação OR (op_sel = 0001) . . . . . 17
Figura 4 – Simulação da operação XOR (op_sel = 0010) . . . . . 17
Figura 5 – Simulação da operação AND (op_sel = 0011) . . . . . 18
Figura 6 – Simulação da operação NOT (op_sel = 1000) . . . . . 18
Figura 7 – Simulação do exemplo 2 de soma: 37h + 19h . . . . . 19
Figura 8 – Simulação do exemplo 4 de soma: 77h + 89h . . . . . 19
Figura 9 – Simulação do exemplo 6 de soma: C7h + D0h . . . . . 19
Figura 10 – Simulação da operação ADDC (op_sel = 0100, c_in = 0) . . . . . 19
Figura 11 – Simulação da operação ADD (op_sel = 0101, c_in = 1) . . . . . 20
Figura 12 – Simulação da operação SUBC (op_sel = 0110, c_in = 1) . . . . . 20
Figura 13 – Simulação da operação SUB (op_sel = 0111, c_in = 1) . . . . . 21
Figura 14 – Simulação da operação SRA (op_sel = 1001) . . . . . 21
Figura 15 – Simulação da operação SRL (op_sel = 1010) . . . . . 21
Figura 16 – Simulação da operação SLL (op_sel = 1011) . . . . . 22
Figura 17 – Simulação da operação RRC (op_sel = 1100, c_in = 0) . . . . . 22
Figura 18 – Simulação da operação RLC (op_sel = 1101, c_in = 0) . . . . . 22
Figura 19 – Simulação da operação RR (op_sel = 1110) . . . . . 23

# LISTA DE TABELAS

Tabela 1 – Entradas e saídas da ULA . . . . . 7
Tabela 2 – Sinalizadores da saída `flags_out` . . . . . 8
Tabela 3 – Operações da ULA . . . . . 8
Tabela 4 – Expressões VHDL de NOT, deslocamentos e rotações . . . . . 12
Tabela 5 – Casos de teste das operações lógicas . . . . . 16
Tabela 6 – Exemplos de soma (ADD) do enunciado . . . . . 18
Tabela 7 – Casos de teste de ADDC, ADD, SUBC e SUB . . . . . 20
Tabela 8 – Casos de teste de SRA, SRL e SLL . . . . . 21
Tabela 9 – Casos de teste de RRC, RLC e RR . . . . . 22

# SUMÁRIO

**1 Introdução . . . . . 6**
**2 Especificação da ULA . . . . . 7**
2.1 Entradas e saídas . . . . . 7
2.2 Sinalizadores (*flags*) . . . . . 7
2.3 Operações . . . . . 7
**3 Desenvolvimento do projeto . . . . . 9**
3.1 Ambiente e restrições . . . . . 9
3.2 Entidade . . . . . 9
3.3 Estratégia geral . . . . . 9
3.4 Operações lógicas . . . . . 10
3.5 Operações aritméticas . . . . . 10
3.6 NOT, deslocamentos e rotações . . . . . 12
3.7 Seleção do resultado e sinalizadores . . . . . 13
**4 Casos de teste e resultados . . . . . 15**
4.1 Metodologia . . . . . 15
4.2 Compilação e síntese . . . . . 15
4.3 Operações lógicas . . . . . 15
4.4 Soma: exemplos do enunciado . . . . . 15
4.5 Demais operações aritméticas . . . . . 16
4.6 Deslocamentos . . . . . 17
4.7 Rotações . . . . . 18
**5 Conclusão . . . . . 24**

**Referências . . . . . 25**

**Apêndices 26**
**APÊNDICE A Código VHDL completo da ULA . . . . . 27**

# 1 INTRODUÇÃO

Este trabalho tem como objetivo o desenvolvimento de uma unidade lógica e aritmética (ULA, ou *ALU*, na sigla em inglês) de 8 bits. O projeto consiste em descrever o circuito em linguagem VHDL, comentar o código, simular o funcionamento e descrever os casos de teste e os resultados da simulação, de acordo com as especificações apresentadas no enunciado (GARCIA, 2026).

Conforme exigido, a descrição utiliza exclusivamente código concorrente, e todas as entradas e saídas são dos tipos `std_logic` ou `std_logic_vector`. O projeto é de lógica combinacional, ou seja, não possui *latches* nem *flip-flops*. A ferramenta utilizada foi o Quartus II, versão 9.1sp2, conforme determinado.

Os conceitos de linguagem, de código concorrente e o exemplo de ULA utilizados como base foram os apresentados em aula (GARCIA; MAGALHÃES, 2024; GARCIA; MAGALHÃES, 2023; GARCIA; MAGALHÃES, 2025a; GARCIA; MAGALHÃES, 2025b).

O restante do documento está organizado da seguinte forma: o Capítulo 2 resume a especificação da ULA; o Capítulo 3 explica as decisões de projeto e o código VHDL; o Capítulo 4 apresenta a metodologia de simulação, os casos de teste e os resultados; e o Capítulo 5 traz as conclusões. O código completo e comentado está no Apêndice A.

# 2 ESPECIFICAÇÃO DA ULA

## 2.1 ENTRADAS E SAÍDAS

A ULA opera sobre palavras de 8 bits e realiza 16 funções distintas, selecionadas pela entrada `op_sel`. A Tabela 1 relaciona os sinais do enunciado com os nomes das portas usados no código VHDL.

Tabela 1 – Entradas e saídas da ULA

| Sinal do enunciado | Porta no VHDL | Direção | Tipo e função |
|---|---|---|---|
| a_in[7..0] | `entrada1` | entrada | `STD_LOGIC_VECTOR(7 DOWNTO 0)`; operando A |
| b_in[7..0] | `entrada2` | entrada | `STD_LOGIC_VECTOR(7 DOWNTO 0)`; operando B |
| c_in | `carry` | entrada | `STD_LOGIC`; *carry* de entrada |
| op_sel[3..0] | `operacao` | entrada | `STD_LOGIC_VECTOR(3 DOWNTO 0)`; seleção da operação |
| r_out[7..0] | `saida` | saída | `STD_LOGIC_VECTOR(7 DOWNTO 0)`; resultado |
| flags_out[4..0] | `flags` | saída | `STD_LOGIC_VECTOR(4 DOWNTO 0)`; sinalizadores |

Fonte: Adaptada de Garcia (2026).

## 2.2 SINALIZADORES (*FLAGS*)

A saída `flags_out` reúne cinco sinalizadores, descritos na Tabela 2.

## 2.3 OPERAÇÕES

A Tabela 3 resume as 16 operações. Em todas elas, `z_flag` vale ‘1’ quando `r_out` é zero e `n_flag` é igual a `r_out[7]`. A sigla “X” indica “não interessa” (*don’t care*).

Tabela 2 – Sinalizadores da saída `flags_out`

| Bit | Sinal | Nome | Significado |
|---|---|---|---|
| 0 | n_flag | *negative* | Bit 7 do resultado. |
| 1 | v_flag | *overflow* | Indica *overflow* nas operações de soma e subtração. |
| 2 | z_flag | *zero* | Vale '1' quando o resultado é zero. |
| 3 | h_flag | *half carry/borrow* | Na soma, é o *carry* do bit 3 para o bit 4; na subtração, é o *borrow* (empréstimo) correspondente. |
| 4 | c_flag | *carry/borrow* | Na soma, é o *carry out* do bit mais significativo; na subtração, é o *borrow out*. Também é usado nas operações de rotação e de deslocamento. |

Fonte: Adaptada de Garcia (2026).

Tabela 3 – Operações da ULA

| op_sel | Mnemônico | r_out | c_flag | h_flag | v_flag |
|---|---|---|---|---|---|
| 0000 | PASS_B | b_in | 0 | 0 | X |
| 0001 | OR | a_in OR b_in | 0 | 0 | X |
| 0010 | XOR | a_in XOR b_in | 0 | 0 | X |
| 0011 | AND | a_in AND b_in | 0 | 0 | X |
| 0100 | ADDC | a_in + b_in + c_in | *carry* | *carry* 3→4 | *overflow* |
| 0101 | ADD | a_in + b_in | *carry* | *carry* 3→4 | *overflow* |
| 0110 | SUBC | a_in − b_in − c_in | *borrow* | *borrow* 3→4 | *overflow* |
| 0111 | SUB | a_in − b_in | *borrow* | *borrow* 3→4 | *overflow* |
| 1000 | NOT | NOT a_in | 0 | X | X |
| 1001 | SRA | a_in[7], a_in[7..1] | a_in[0] | X | X |
| 1010 | SRL | '0', a_in[7..1] | a_in[0] | X | X |
| 1011 | SLL | a_in[6..0], '0' | a_in[7] | X | X |
| 1100 | RRC | c_in, a_in[7..1] | a_in[0] | X | X |
| 1101 | RLC | a_in[6..0], c_in | a_in[7] | X | X |
| 1110 | RR | a_in[0], a_in[7..1] | a_in[0] | X | X |
| 1111 | RL | a_in[6..0], a_in[7] | a_in[7] | X | X |

Fonte: Adaptada de Garcia (2026).

# 3 DESENVOLVIMENTO DO PROJETO

## 3.1 AMBIENTE E RESTRIÇÕES

O projeto foi desenvolvido no Quartus II 9.1sp2, para o dispositivo EP2C35F672C6 da família Cyclone II. A entidade chama-se `Trabalho1_ULA` e a arquitetura, `ula`. Todo o código é concorrente: não há `PROCESS`, `IF` nem `CASE`, apenas atribuições concorrentes e operadores, além das construções `WITH...SELECT` e `WHEN...ELSE`. Todas as portas são `STD_LOGIC` ou `STD_LOGIC_VECTOR`. A biblioteca `numeric_std` é usada apenas para realizar somas e subtrações por meio de conversões para `UNSIGNED`, sem alterar os tipos das portas.

## 3.2 ENTIDADE

O Código 3.1 mostra a declaração da entidade, com as portas descritas na Tabela 1.

Código 3.1 – Declaração da entidade

```
5  ENTITY Trabalho1_ULA IS
6    PORT(
7      entrada1, entrada2: IN STD_LOGIC_VECTOR(7 DOWNTO 0);
8      carry: IN STD_LOGIC;
9      operacao: IN STD_LOGIC_VECTOR(3 DOWNTO 0);
10     saida: OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
11     flags: OUT STD_LOGIC_VECTOR(4 DOWNTO 0)
12   );
13 END ENTITY;
```

## 3.3 ESTRATÉGIA GERAL

Cada operação é calculada em paralelo, em um sinal próprio (por exemplo, `op_or`, `op_add`, `op_rrc`). Em seguida, um multiplexador descrito com `WITH...SELECT` escolhe, a partir de `operacao`, qual sinal vai para o resultado. O mesmo método é usado para escolher o *carry*, o *half carry* e o *overflow*. Os sinalizadores `n` e `z` são calculados diretamente a partir do resultado selecionado. Como todas as operações ficam sempre ativas e a seleção é feita por um multiplexador, o circuito resultante é puramente combinacional.

## 3.4 OPERAÇÕES LÓGICAS

As operações `PASS_B`, `OR`, `XOR` e `AND` são atribuições diretas com o operador correspondente (Código 3.2). A operação `NOT` segue a mesma ideia e está no Código 3.4.

Código 3.2 – Operações lógicas

```
61  --primeira operacao 0000 entrada de B vai para a saida
62  pass_b <= entrada2;
63
64  --segunda operacao 0001 a saida e A or B
65  op_or <= entrada1 OR entrada2;
66
67  --terceira operacao 0010 a saida e A xor B
68  op_xor <= entrada1 XOR entrada2;
69
70  --quarta operacao 0011 a saida e A and B
71  op_and <= entrada1 AND entrada2;
```

## 3.5 OPERAÇÕES ARITMÉTICAS

As quatro operações aritméticas (ADDC, ADD, SUBC e SUB) seguem a mesma técnica. Os operandos são estendidos com um bit '0' à esquerda (9 bits), de modo que o bit 8 da soma ou da subtração seja o *carry out* ou o *borrow out*. O resultado de 8 bits é formado pelos bits 7 a 0. O *half carry/borrow* é obtido da mesma forma, com os 4 bits menos significativos dos operandos estendidos para 5 bits: o bit 4 do resultado é o *carry* ou o *borrow* do bit 3 para o bit 4. Nas operações com *carry in*, o sinal `carry` é estendido e somado ou subtraído como um terceiro operando.

O *overflow* é detectado pelos sinais dos operandos e do resultado. Na soma, ocorre quando dois operandos positivos geram resultado negativo ou quando dois negativos geram resultado positivo (Equação 3.1). Na subtração, ocorre quando um positivo menos um negativo gera resultado negativo ou quando um negativo menos um positivo gera resultado positivo (Equação 3.2).

$$v_{soma} = a_7 \cdot b_7 \cdot \overline{r_7} + \overline{a_7} \cdot \overline{b_7} \cdot r_7 \tag{3.1}$$

$$v_{sub} = a_7 \cdot \overline{b_7} \cdot \overline{r_7} + \overline{a_7} \cdot b_7 \cdot r_7 \tag{3.2}$$

O Código 3.3 mostra as quatro operações.

Código 3.3 – Operações aritméticas

```
73  --quinta operacao 0100 a saida e A+B+carry
74  --primeiramente precisamos estender o tamanho do sinal de carry para podermos
        somar
75  aux_carry_1 <= "00000000" & carry;
76  aux_carry_2 <= "0000" & carry;
77  -- operandos estendidos com '0' & para o carry nao se perder
78  aux_somac_1 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1)+UNSIGNED('0' &
        entrada2)+UNSIGNED(aux_carry_1));
79  aux_somac_2 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1(3 DOWNTO 0))+UNSIGNED
        ('0' & entrada2(3 DOWNTO 0))+UNSIGNED(aux_carry_2));
80  op_addc <= aux_somac_1(7 DOWNTO 0);
81  carry_flag_addc <= aux_somac_1(8);
82  halfcarry_flag_addc <= aux_somac_2(4);
83  overflow_flag_addc <= (entrada1(7) and entrada2(7) and not op_addc(7)) or (
        not entrada1(7) and not entrada2(7) and op_addc(7));
84  --fim soma com carry
85
86  --soma sem carry
87  aux_soma_1 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1)+UNSIGNED('0' &
        entrada2));
88  aux_soma_2 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1(3 DOWNTO 0))+UNSIGNED
        ('0' & entrada2(3 DOWNTO 0)));
89  op_add <= aux_soma_1(7 DOWNTO 0);
90  carry_flag_add <= aux_soma_1(8);
91  halfcarry_flag_add <= aux_soma_2(4);
92  overflow_flag_add <= (entrada1(7) and entrada2(7) and not op_add(7)) or (not
        entrada1(7) and not entrada2(7) and op_add(7));
93  --fim soma sem carry
94
95  --subtracao com carry
96  aux_subc_1 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1)-UNSIGNED('0' &
        entrada2)-UNSIGNED(aux_carry_1));
97  aux_subc_2 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1(3 DOWNTO 0))-UNSIGNED
        ('0' & entrada2(3 DOWNTO 0))-UNSIGNED(aux_carry_2));
98  op_subc <= aux_subc_1(7 DOWNTO 0);
99  carry_flag_subc <= aux_subc_1(8);
100 halfcarry_flag_subc <= aux_subc_2(4);
101 overflow_flag_subc <= (entrada1(7) and not entrada2(7) and not op_subc(7)) or
        (not entrada1(7) and entrada2(7) and op_subc(7));
102 --fim subtracao com carry
103
104 --subtracao sem carry
105 aux_sub_1 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1)-UNSIGNED('0' &
        entrada2));
106 aux_sub_2 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1(3 DOWNTO 0))-UNSIGNED
        ('0' & entrada2(3 DOWNTO 0)));
107 op_sub <= aux_sub_1(7 DOWNTO 0);
```

```
108   carry_flag_sub <= aux_sub_1(8);
109   halfcarry_flag_sub <= aux_sub_2(4);
110   overflow_flag_sub <= (entrada1(7) and not entrada2(7) and not op_sub(7)) or (
          not entrada1(7) and entrada2(7) and op_sub(7));
111   --fim subtracao sem carry
```

## 3.6 NOT, DESLOCAMENTOS E ROTAÇÕES

Os deslocamentos e as rotações são feitos por concatenação (operador `&`) de parte dos bits de `entrada1` com um bit novo. O bit que sai da palavra é levado para o *carry*. Por exemplo, em `SRL` entra '0' à esquerda e `entrada1(0)` vai para o *carry*; em `RRC` o *carry* de entrada ocupa o bit 7 e `entrada1(0)` se torna o novo *carry*. A Tabela 4 resume as expressões usadas.

Tabela 4 – Expressões VHDL de NOT, deslocamentos e rotações

| op_sel | Mnemônico | Resultado | *Carry* |
|---|---|---|---|
| 1000 | NOT | `NOT entrada1` | '0' |
| 1001 | SRA | `entrada1(7) & entrada1(7 DOWNTO 1)` | `entrada1(0)` |
| 1010 | SRL | `'0' & entrada1(7 DOWNTO 1)` | `entrada1(0)` |
| 1011 | SLL | `entrada1(6 DOWNTO 0) & '0'` | `entrada1(7)` |
| 1100 | RRC | `carry & entrada1(7 DOWNTO 1)` | `entrada1(0)` |
| 1101 | RLC | `entrada1(6 DOWNTO 0) & carry` | `entrada1(7)` |
| 1110 | RR | `entrada1(0) & entrada1(7 DOWNTO 1)` | `entrada1(0)` |
| 1111 | RL | `entrada1(6 DOWNTO 0) & entrada1(7)` | `entrada1(7)` |

Fonte: Elaborada pelos autores (2026).

Código 3.4 – NOT, deslocamentos e rotações

```
113   --nona operacao 1000 a saida e NOT A (carry = '0')
114   op_not <= NOT entrada1;
115
116   --decima operacao 1001 SRA: deslocamento aritmetico para a direita
117   --o bit 7 e repetido e o bit 0 vai para o carry
118   op_sra <= entrada1(7) & entrada1(7 DOWNTO 1);
119   carry_flag_sra <= entrada1(0);
120
121   --decima primeira operacao 1010 SRL: deslocamento logico para a direita
122   --entra '0' no bit 7 e o bit 0 vai para o carry
123   op_srl <= '0' & entrada1(7 DOWNTO 1);
124   carry_flag_srl <= entrada1(0);
125
126   --decima segunda operacao 1011 SLL: deslocamento logico para a esquerda
127   --entra '0' no bit 0 e o bit 7 vai para o carry
128   op_sll <= entrada1(6 DOWNTO 0) & '0';
129   carry_flag_sll <= entrada1(7);
```

```
130
131   --decima terceira operacao 1100 RRC: rotacao para a direita atraves do carry
132   --o carry de entrada vai para o bit 7 e o bit 0 vai para o carry de saida
133   op_rrc <= carry & entrada1(7 DOWNTO 1);
134   carry_flag_rrc <= entrada1(0);
135
136   --decima quarta operacao 1101 RLC: rotacao para a esquerda atraves do carry
137   --o carry de entrada vai para o bit 0 e o bit 7 vai para o carry de saida
138   op_rlc <= entrada1(6 DOWNTO 0) & carry;
139   carry_flag_rlc <= entrada1(7);
140
141   --decima quinta operacao 1110 RR: rotacao para a direita
142   --o bit 0 vai para o bit 7 e tambem para o carry
143   op_rr <= entrada1(0) & entrada1(7 DOWNTO 1);
144   carry_flag_rr <= entrada1(0);
145
146   --decima sexta operacao 1111 RL: rotacao para a esquerda
147   --o bit 7 vai para o bit 0 e tambem para o carry
148   op_rl <= entrada1(6 DOWNTO 0) & entrada1(7);
149   carry_flag_rl <= entrada1(7);
```

## 3.7 SELEÇÃO DO RESULTADO E SINALIZADORES

O Código 3.5 mostra os multiplexadores de seleção e a atribuição das saídas. Nas operações lógicas e em NOT, o *carry* vale '0', como pede o enunciado. Nas operações em que o enunciado indica "não interessa" (*overflow* nas operações lógicas, deslocamentos e rotações; *half carry* em NOT, deslocamentos e rotações), a cláusula WHEN OTHERS fornece '0'. Os sinalizadores n e z valem, respectivamente, o bit 7 do resultado e '1' quando o resultado é zero.

Código 3.5 – Seleção do resultado e sinalizadores

```
151   WITH operacao SELECT
152     resultado <= pass_b WHEN "0000",
153            op_or WHEN "0001",
154            op_xor WHEN "0010",
155            op_and WHEN "0011",
156            op_addc WHEN "0100",
157            op_add WHEN "0101",
158            op_subc WHEN "0110",
159            op_sub WHEN "0111",
160            op_not WHEN "1000",
161            op_sra WHEN "1001",
162            op_srl WHEN "1010",
163            op_sll WHEN "1011",
164            op_rrc WHEN "1100",
```

```
165             op_rlc WHEN "1101",
166             op_rr WHEN "1110",
167             op_rl WHEN "1111",
168             "00000000" WHEN OTHERS;
169
170   --nas operacoes logicas (0000 a 0011 e 1000) o carry e '0'
171   WITH operacao SELECT
172     carry_flag <= carry_flag_addc WHEN "0100",
173             carry_flag_add WHEN "0101",
174             carry_flag_subc WHEN "0110",
175             carry_flag_sub WHEN "0111",
176             carry_flag_sra WHEN "1001",
177             carry_flag_srl WHEN "1010",
178             carry_flag_sll WHEN "1011",
179             carry_flag_rrc WHEN "1100",
180             carry_flag_rlc WHEN "1101",
181             carry_flag_rr WHEN "1110",
182             carry_flag_rl WHEN "1111",
183             '0' WHEN OTHERS;
184
185   --halfcarry: so tem significado nas operacoes aritmeticas (nas demais e don't
        care, usamos '0')
186   WITH operacao SELECT
187     halfcarry_flag <= halfcarry_flag_addc WHEN "0100",
188               halfcarry_flag_add WHEN "0101",
189               halfcarry_flag_subc WHEN "0110",
190               halfcarry_flag_sub WHEN "0111",
191               '0' WHEN OTHERS;
192
193   --overflow: so tem significado nas operacoes aritmeticas (nas demais e don't
       care, usamos '0')
194   WITH operacao SELECT
195     overflow_flag <= overflow_flag_addc WHEN "0100",
196               overflow_flag_add WHEN "0101",
197               overflow_flag_subc WHEN "0110",
198               overflow_flag_sub WHEN "0111",
199               '0' WHEN OTHERS;
200
201   saida <= resultado;
202
203   flags(0) <= resultado(7);                              -- negative
204   flags(1) <= overflow_flag;                             -- overflow
205   flags(2) <= '1' WHEN resultado = "00000000" ELSE '0'; -- zero
206   flags(3) <= halfcarry_flag;                            -- half carry / borrow
207   flags(4) <= carry_flag;                                -- carry / borrow
```

# 4 CASOS DE TESTE E RESULTADOS

## 4.1 METODOLOGIA

A validação foi feita por simulação funcional no simulador do Quartus II 9.1sp2, com um arquivo de formas de onda vetoriais (`Trabalho1_ULA.vwf`) para o dispositivo EP2C35F672C6. Em cada simulação a operação (`operacao`) e o *carry* ficam fixos, e as entradas `entrada1` e `entrada2` assumem dois valores: 00h em ambas, de 0 a 10 ns, que exercita o resultado zero; e um par de valores diferentes de zero, de 10 a 40 ns, cujos resultados são lidos na coluna "Value at 30 ns" do simulador. Para a maioria das operações o par é 77h e 89h; para os exemplos de soma do enunciado, os pares são os do próprio enunciado. A RRC foi simulada apenas com o par 77h e 89h.

Os valores esperados foram calculados a partir da especificação (Capítulo 2). Nas tabelas deste capítulo, `a_in`, `b_in` e `r_out` estão em hexadecimal; `c_in` e os sinalizadores estão em binário. Os cinco sinalizadores aparecem juntos, na ordem de `flags_out[4..0]`: `c` (*carry/borrow*), `h` (*half carry/borrow*), `z` (*zero*), `v` (*overflow*) e `n` (*negative*). Por exemplo, 11100 indica `c` = 1, `h` = 1, `z` = 1, `v` = 0 e `n` = 0. Nas operações em que o enunciado indica "não interessa", o circuito entrega '0'.

## 4.2 COMPILAÇÃO E SÍNTESE

A Figura 1 mostra o relatório de compilação (*Flow Summary*) do Quartus II 9.1sp2: o processo terminou com sucesso. O projeto usa 135 elementos lógicos, todos funções combinacionais (menos de 1% do dispositivo), e nenhum registrador, o que confirma que o circuito é puramente combinacional, sem *latches* nem *flip-flops*. Os pinos usados são as 34 portas da entidade (8 + 8 + 1 + 4 de entrada e 8 + 5 de saída).

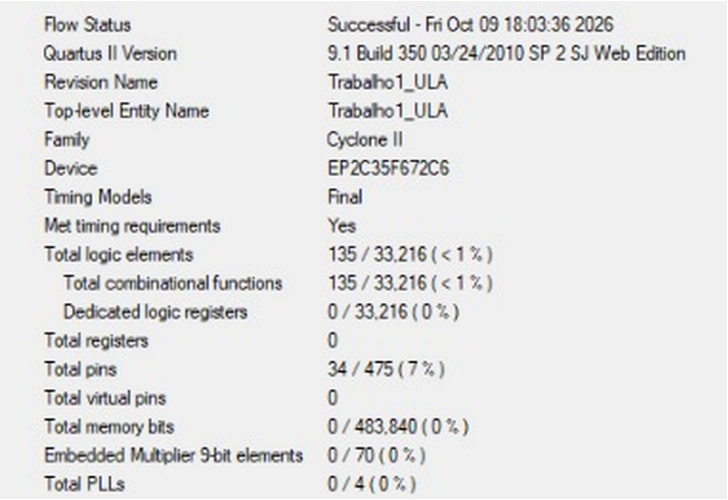
## 4.3 OPERAÇÕES LÓGICAS

A Tabela 5 apresenta os casos de teste de PASS_B, OR, XOR, AND e NOT, com `c_in` = 0. Os valores obtidos nas Figuras 2 a 6 coincidem com os esperados em todos os casos.

## 4.4 SOMA: EXEMPLOS DO ENUNCIADO

O enunciado fornece seis exemplos para a operação ADD. Foram simulados os exemplos 2, 4 e 6 (Tabela 6), que cobrem *half carry* (exemplo 2), *carry* com resultado zero (exemplo 4) e

Figura 1 – Relatório de compilação (*Flow Summary*) no Quartus II 9.1sp2



| | |
|---|---|
| Flow Status | Successful - Fri Oct 09 18:03:36 2026 |
| Quartus II Version | 9.1 Build 350 03/24/2010 SP 2 SJ Web Edition |
| Revision Name | Trabalho1_ULA |
| Top-level Entity Name | Trabalho1_ULA |
| Family | Cyclone II |
| Device | EP2C35F672C6 |
| Timing Models | Final |
| Met timing requirements | Yes |
| Total logic elements | 135 / 33,216 ( < 1 % ) |
| Total combinational functions | 135 / 33,216 ( < 1 % ) |
| Dedicated logic registers | 0 / 33,216 ( 0 % ) |
| Total registers | 0 |
| Total pins | 34 / 475 ( 7 % ) |
| Total virtual pins | 0 |
| Total memory bits | 0 / 483,840 ( 0 % ) |
| Embedded Multiplier 9-bit elements | 0 / 70 ( 0 % ) |
| Total PLLs | 0 / 4 ( 0 % ) |

Fonte: Elaborada pelos autores (2026).

Tabela 5 – Casos de teste das operações lógicas

| | | | | | | Esperado | | Obtido | |
|---|---|---|---|---|---|---|---|---|---|
| Nº | Operação | op_sel | a_in | b_in | c_in | r_out | flags | r_out | flags |
| 1 | PASS_B | 0000 | 00 | 00 | 0 | 00 | 00100 | 00 | 00100 |
| 2 | PASS_B | 0000 | 77 | 89 | 0 | 89 | 00001 | 89 | 00001 |
| 3 | OR | 0001 | 00 | 00 | 0 | 00 | 00100 | 00 | 00100 |
| 4 | OR | 0001 | 77 | 89 | 0 | FF | 00001 | FF | 00001 |
| 5 | XOR | 0010 | 00 | 00 | 0 | 00 | 00100 | 00 | 00100 |
| 6 | XOR | 0010 | 77 | 89 | 0 | FE | 00001 | FE | 00001 |
| 7 | AND | 0011 | 00 | 00 | 0 | 00 | 00100 | 00 | 00100 |
| 8 | AND | 0011 | 77 | 89 | 0 | 01 | 00000 | 01 | 00000 |
| 9 | NOT | 1000 | 00 | 00 | 0 | FF | 00001 | FF | 00001 |
| 10 | NOT | 1000 | 77 | 89 | 0 | 88 | 00001 | 88 | 00001 |

Fonte: Elaborada pelos autores (2026). Valores em hexadecimal (a_in, b_in e r_out) e binário (c_in e flags, na ordem flags_out[4..0] = c, h, z, v, n).

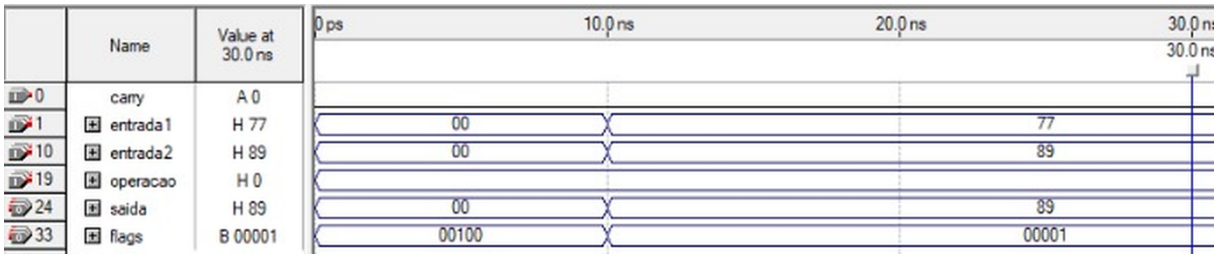
*carry* com resultado negativo (exemplo 6).

As Figuras 7, 8 e 9 mostram as simulações. Os valores obtidos coincidem com os esperados e com os do enunciado.

## 4.5 DEMAIS OPERAÇÕES ARITMÉTICAS

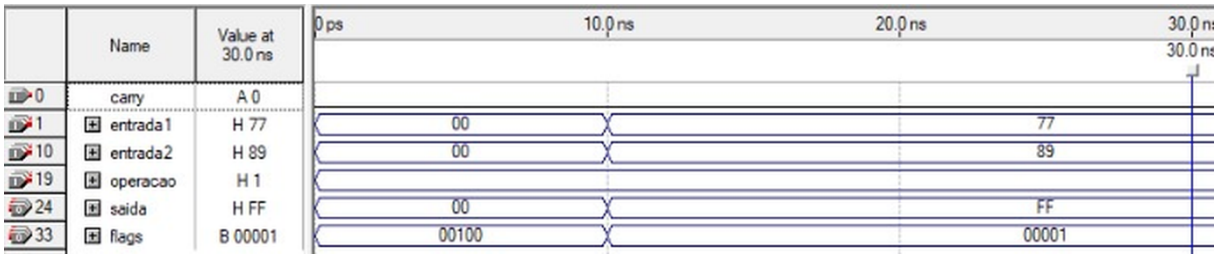
A Tabela 7 reúne os casos de ADDC, ADD (com `c_in` = 1), SUBC e SUB (com `c_in` = 1). A soma 77h + 89h gera *carry*, *half carry* e resultado zero; as subtrações 00h − 00h − 1 e 77h − 89h geram *borrow* e, no segundo caso, também *overflow* (um número positivo menos um negativo com resultado negativo). Nos casos de ADD e SUB, `c_in` = 1 é aplicado de propósito,

Figura 2 – Simulação da operação PASS_B (op_sel = 0000)



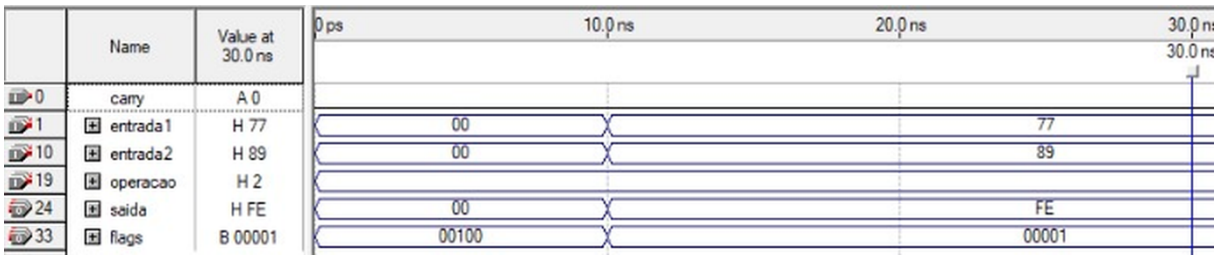

Fonte: Elaborada pelos autores (2026).

Figura 3 – Simulação da operação OR (op_sel = 0001)




Fonte: Elaborada pelos autores (2026).

Figura 4 – Simulação da operação XOR (op_sel = 0010)



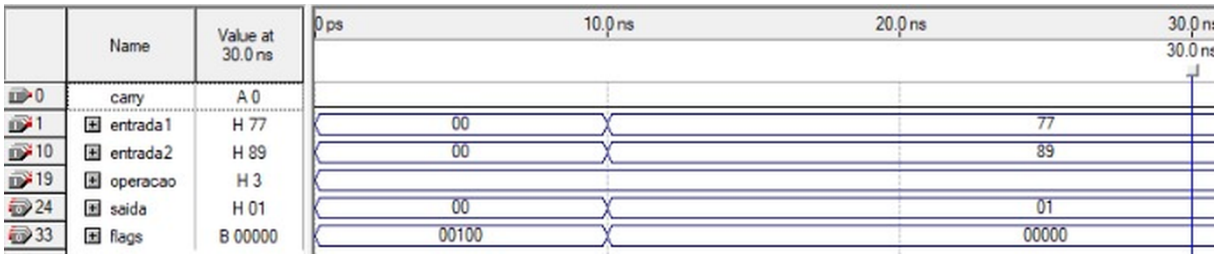

Fonte: Elaborada pelos autores (2026).

para mostrar que essas operações o ignoram. Os valores obtidos nas Figuras 10 a 13 coincidem com os esperados em todos os casos.

## 4.6 DESLOCAMENTOS

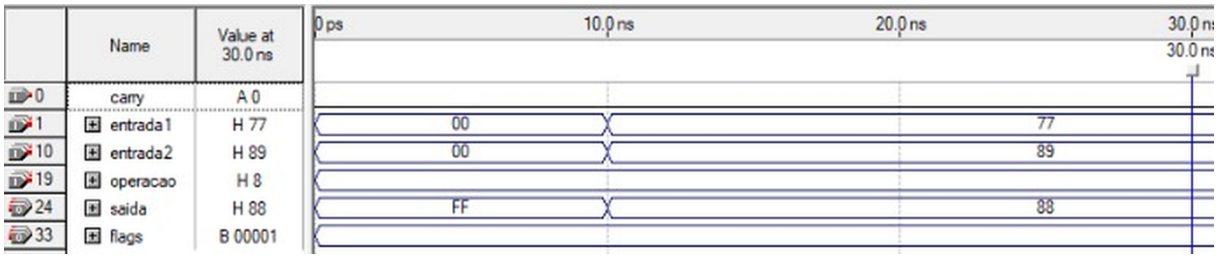
A Tabela 8 reúne os casos de SRA, SRL e SLL. Com `a_in` = 77h, SRA e SRL geram 3Bh com *carry* = 1 (o bit 0 que saiu), e SLL gera EEh com *carry* = 0 (o bit 7 que saiu). Os valores obtidos nas Figuras 14 a 16 coincidem com os esperados.

Figura 5 – Simulação da operação AND (op_sel = 0011)




Fonte: Elaborada pelos autores (2026).

Figura 6 – Simulação da operação NOT (op_sel = 1000)




Fonte: Elaborada pelos autores (2026).

Tabela 6 – Exemplos de soma (ADD) do enunciado

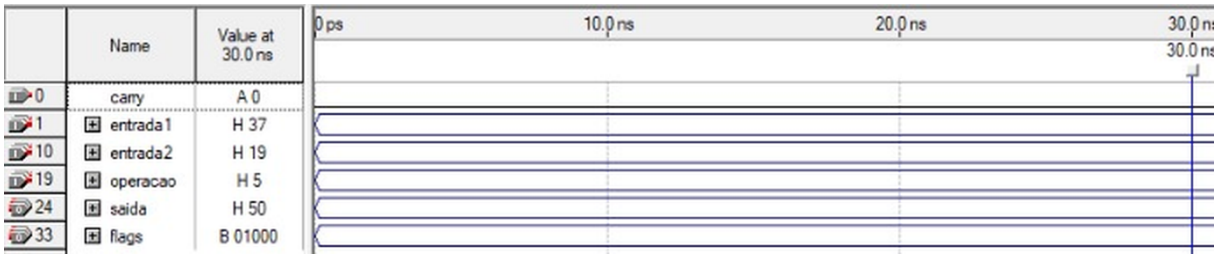
| | | | | | | Esperado | | Obtido | |
|---|---|---|---|---|---|---|---|---|---|
| Nº | Operação | op_sel | a_in | b_in | c_in | r_out | flags | r_out | flags |
| 2 | ADD | 0101 | 37 | 19 | 0 | 50 | 01000 | 50 | 01000 |
| 4 | ADD | 0101 | 77 | 89 | 0 | 00 | 11100 | 00 | 11100 |
| 6 | ADD | 0101 | C7 | D0 | 0 | 97 | 10001 | 97 | 10001 |

Fonte: Elaborada pelos autores (2026). Valores em hexadecimal (a_in, b_in e r_out) e binário (c_in e flags, na ordem flags_out[4..0] = c, h, z, v, n).

## 4.7 ROTAÇÕES

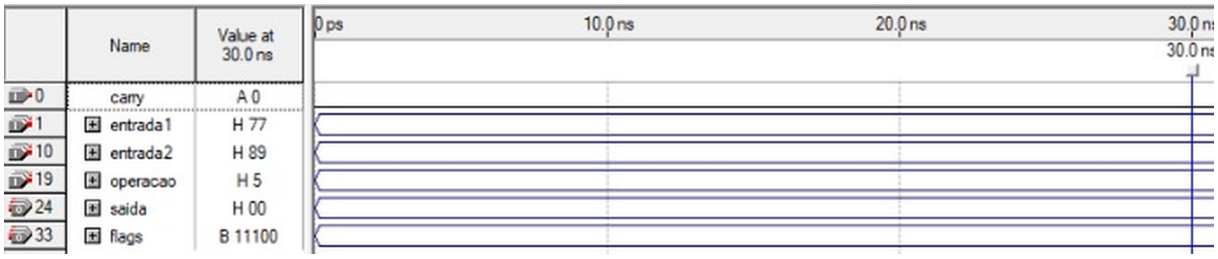
A Tabela 9 reúne os casos de RRC, RLC e RR, todos com `c_in` = 0. Os valores obtidos nas Figuras 17, 18 e 19 coincidem com os esperados.

Figura 7 – Simulação do exemplo 2 de soma: 37h + 19h



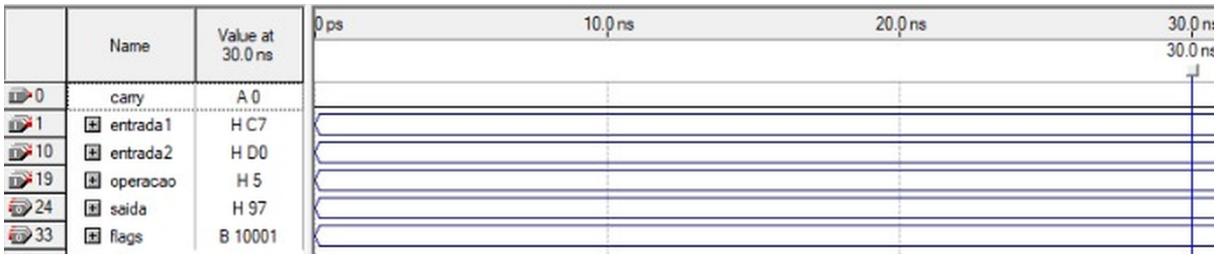

Fonte: Elaborada pelos autores (2026).

Figura 8 – Simulação do exemplo 4 de soma: 77h + 89h



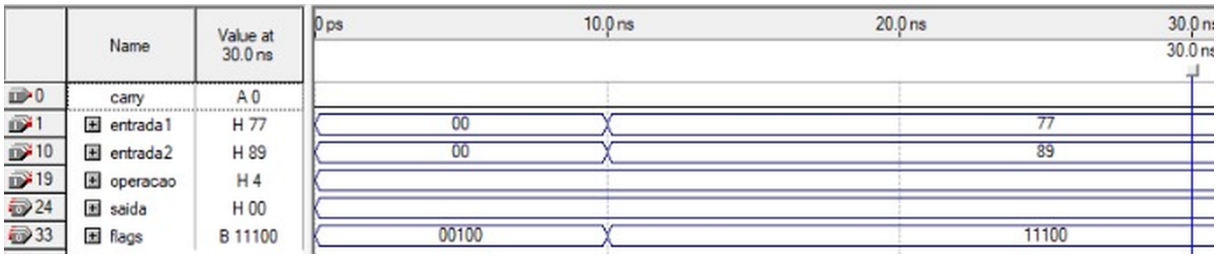

Fonte: Elaborada pelos autores (2026).

Figura 9 – Simulação do exemplo 6 de soma: C7h + D0h




Fonte: Elaborada pelos autores (2026).

Figura 10 – Simulação da operação ADDC (op_sel = 0100, c_in = 0)



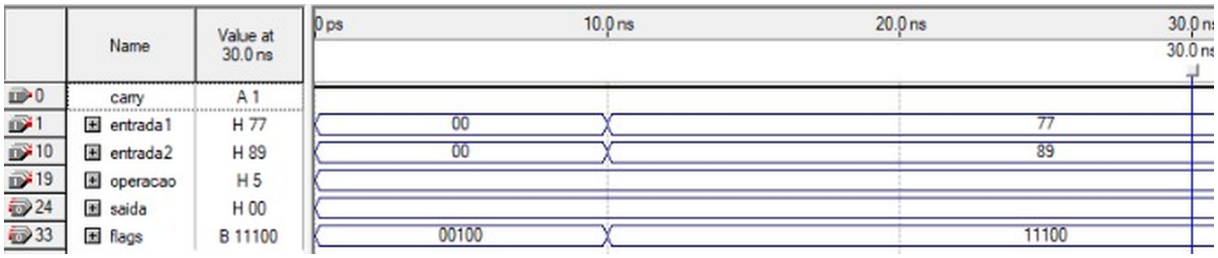

Fonte: Elaborada pelos autores (2026).

Tabela 7 – Casos de teste de ADDC, ADD, SUBC e SUB

| | | | | | | Esperado | | Obtido | |
|---|---|---|---|---|---|---|---|---|---|
| Nº | Operação | op_sel | a_in | b_in | c_in | r_out | flags | r_out | flags |
| 1 | ADDC | 0100 | 00 | 00 | 0 | 00 | 00100 | 00 | 00100 |
| 2 | ADDC | 0100 | 77 | 89 | 0 | 00 | 11100 | 00 | 11100 |
| 3 | ADD | 0101 | 00 | 00 | 1 | 00 | 00100 | 00 | 00100 |
| 4 | ADD | 0101 | 77 | 89 | 1 | 00 | 11100 | 00 | 11100 |
| 5 | SUBC | 0110 | 00 | 00 | 1 | FF | 11001 | FF | 11001 |
| 6 | SUBC | 0110 | 77 | 89 | 1 | ED | 11011 | ED | 11011 |
| 7 | SUB | 0111 | 00 | 00 | 1 | 00 | 00100 | 00 | 00100 |
| 8 | SUB | 0111 | 77 | 89 | 1 | EE | 11011 | EE | 11011 |

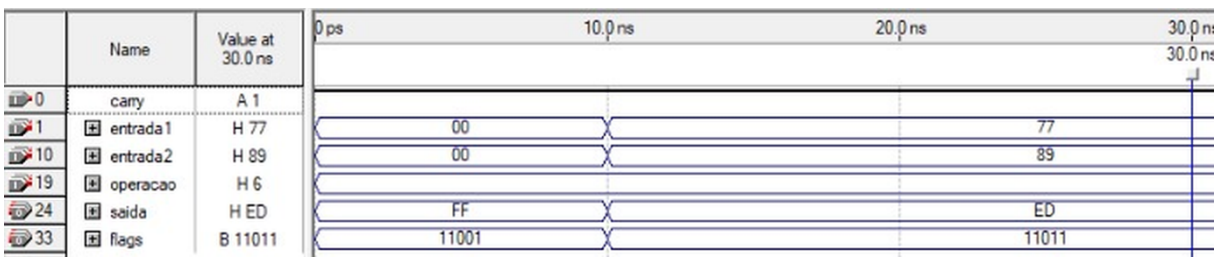
Fonte: Elaborada pelos autores (2026). Valores em hexadecimal (a_in, b_in e r_out) e binário (c_in e flags, na ordem flags_out[4..0] = c, h, z, v, n).

Figura 11 – Simulação da operação ADD (op_sel = 0101, c_in = 1)



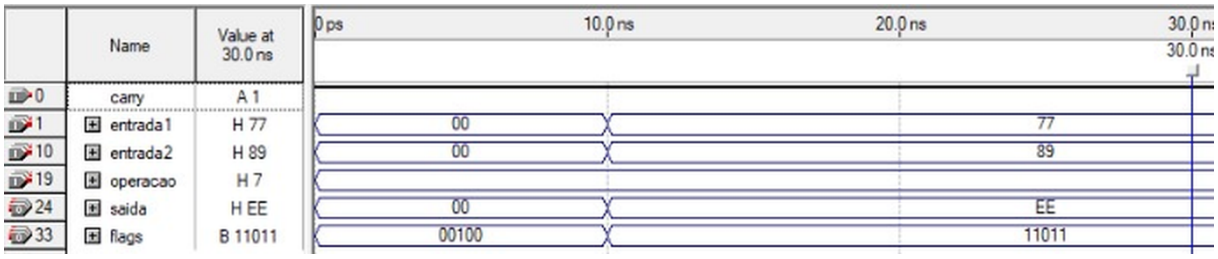

Fonte: Elaborada pelos autores (2026).

Figura 12 – Simulação da operação SUBC (op_sel = 0110, c_in = 1)




Fonte: Elaborada pelos autores (2026).

Figura 13 – Simulação da operação SUB (op_sel = 0111, c_in = 1)



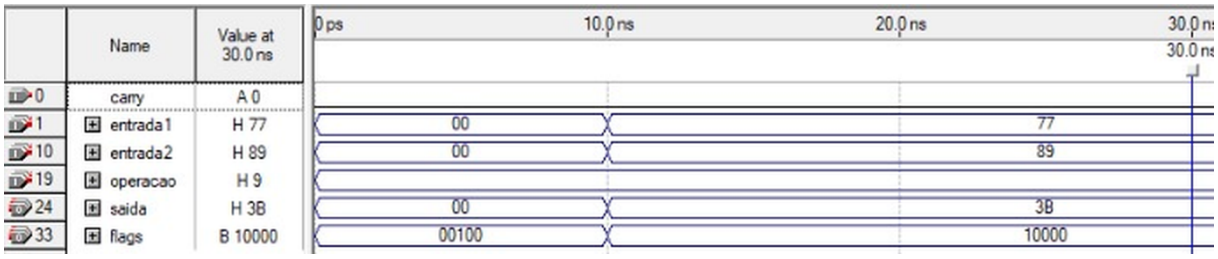

Fonte: Elaborada pelos autores (2026).

Tabela 8 – Casos de teste de SRA, SRL e SLL

| | | | | | | Esperado | | Obtido | |
|---|---|---|---|---|---|---|---|---|---|
| Nº | Operação | op_sel | a_in | b_in | c_in | r_out | flags | r_out | flags |
| 1 | SRA | 1001 | 00 | 00 | 0 | 00 | 00100 | 00 | 00100 |
| 2 | SRA | 1001 | 77 | 89 | 0 | 3B | 10000 | 3B | 10000 |
| 3 | SRL | 1010 | 00 | 00 | 0 | 00 | 00100 | 00 | 00100 |
| 4 | SRL | 1010 | 77 | 89 | 0 | 3B | 10000 | 3B | 10000 |
| 5 | SLL | 1011 | 00 | 00 | 0 | 00 | 00100 | 00 | 00100 |
| 6 | SLL | 1011 | 77 | 89 | 0 | EE | 00001 | EE | 00001 |

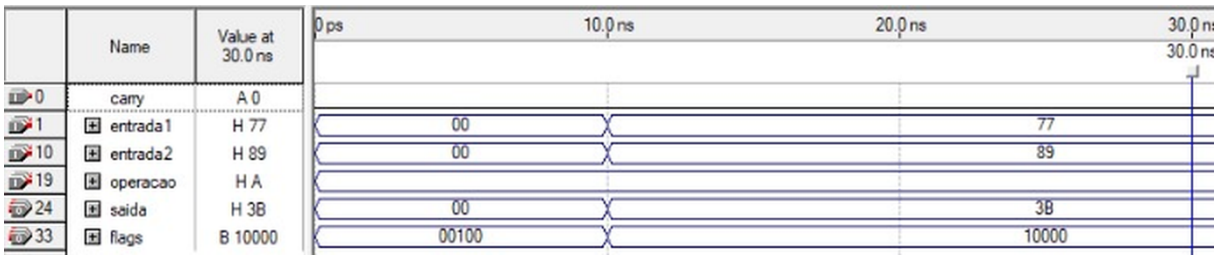
Fonte: Elaborada pelos autores (2026). Valores em hexadecimal (a_in, b_in e r_out) e binário (c_in e flags, na ordem flags_out[4..0] = c, h, z, v, n).

Figura 14 – Simulação da operação SRA (op_sel = 1001)



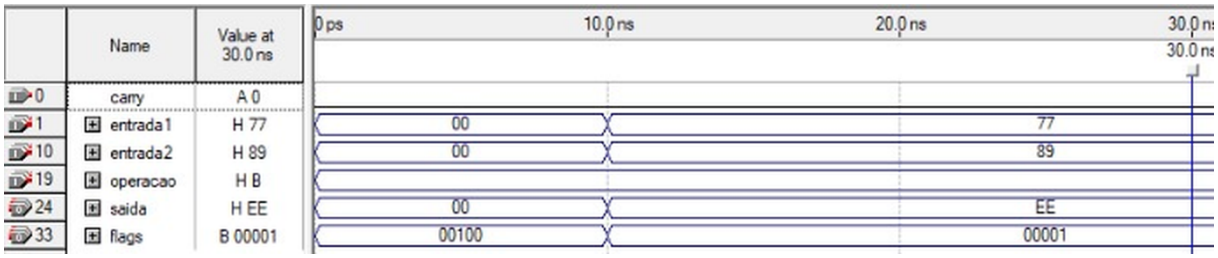

Fonte: Elaborada pelos autores (2026).

Figura 15 – Simulação da operação SRL (op_sel = 1010)




Fonte: Elaborada pelos autores (2026).

Figura 16 – Simulação da operação SLL (op_sel = 1011)



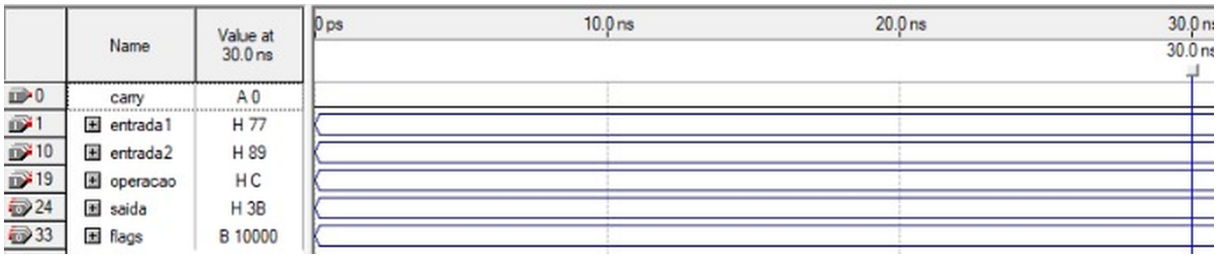

Fonte: Elaborada pelos autores (2026).

Tabela 9 – Casos de teste de RRC, RLC e RR

| | | | | | | Esperado | | Obtido | |
|---|---|---|---|---|---|---|---|---|---|
| Nº | Operação | op_sel | a_in | b_in | c_in | r_out | flags | r_out | flags |
| 1 | RRC | 1100 | 77 | 89 | 0 | 3B | 10000 | 3B | 10000 |
| 2 | RLC | 1101 | 00 | 00 | 0 | 00 | 00100 | 00 | 00100 |
| 3 | RLC | 1101 | 77 | 89 | 0 | EE | 00001 | EE | 00001 |
| 4 | RR | 1110 | 00 | 00 | 0 | 00 | 00100 | 00 | 00100 |
| 5 | RR | 1110 | 77 | 89 | 0 | BB | 10001 | BB | 10001 |

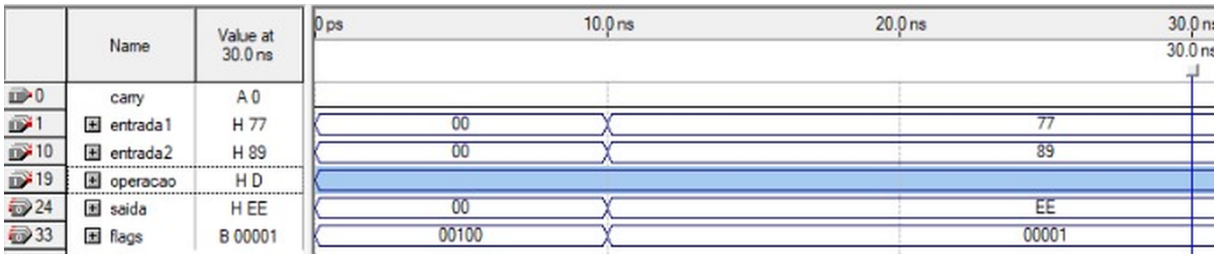
Fonte: Elaborada pelos autores (2026). Valores em hexadecimal (a_in, b_in e r_out) e binário (c_in e flags, na ordem flags_out[4..0] = c, h, z, v, n).

Figura 17 – Simulação da operação RRC (op_sel = 1100, c_in = 0)



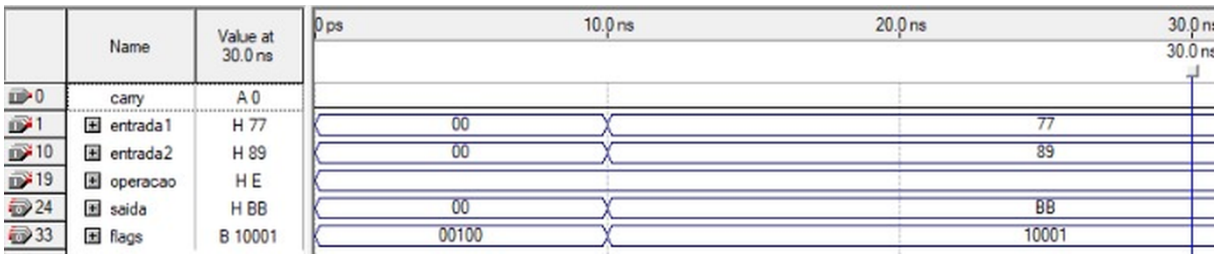

Fonte: Elaborada pelos autores (2026).

Figura 18 – Simulação da operação RLC (op_sel = 1101, c_in = 0)




Fonte: Elaborada pelos autores (2026).

Figura 19 – Simulação da operação RR (op_sel = 1110)




Fonte: Elaborada pelos autores (2026).

# 5 CONCLUSÃO

Foi desenvolvida em VHDL uma ULA de 8 bits com 16 operações e cinco sinalizadores, usando somente código concorrente e portas `std_logic` e `std_logic_vector`, como exigido. A estrutura adotada, com um sinal por operação e multiplexadores `WITH...SELECT`, mantém o circuito puramente combinacional, o que foi confirmado pelo relatório de compilação (nenhum registrador) e facilita a leitura e a verificação do código.

As simulações realizadas apresentaram resultados e sinalizadores iguais aos esperados.

# REFERÊNCIAS

GARCIA, F. *Especificações para o primeiro projeto*. 2026. Material didático. 2.º semestre de 2026.

GARCIA, F.; MAGALHÃES, A. H. *Aula 4 – Linguagem de descrição de hardware (HDL): pacote IEEE-1164 e código concorrente*. 2023. Slides da disciplina Sistemas Reconfiguráveis. Rev. G.

GARCIA, F.; MAGALHÃES, A. H. *Aula 3 – Linguagem de descrição de hardware (HDL): unidades fundamentais, classes, tipos e operadores*. 2024. Slides da disciplina Sistemas Reconfiguráveis. Rev. D.

GARCIA, F.; MAGALHÃES, A. H. *Aula 5 – Exemplos de uso do código concorrente: codificadores com prioridade e multiplexadores*. 2025. Slides da disciplina Sistemas Reconfiguráveis. Rev. C.

GARCIA, F.; MAGALHÃES, A. H. *Aula 6 – Outros tipos. Exemplos de uso do código concorrente: unidade lógica e aritmética (ALU)*. 2025. Slides da disciplina Sistemas Reconfiguráveis. Rev. E.

# Apêndices

# APÊNDICE A – CÓDIGO VHDL COMPLETO DA ULA

Código A.1 – Arquivo Trabalho1_ULA.vhd

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3  USE ieee.numeric_std.all;
4
5  ENTITY Trabalho1_ULA IS
6    PORT(
7      entrada1, entrada2: IN STD_LOGIC_VECTOR(7 DOWNTO 0);
8      carry: IN STD_LOGIC;
9      operacao: IN STD_LOGIC_VECTOR(3 DOWNTO 0);
10     saida: OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
11     flags: OUT STD_LOGIC_VECTOR(4 DOWNTO 0)
12   );
13 END ENTITY;
14
15 ARCHITECTURE ula OF Trabalho1_ULA IS
16   SIGNAL resultado: STD_LOGIC_VECTOR(7 downto 0);
17   SIGNAL pass_b: STD_LOGIC_VECTOR(7 DOWNTO 0);
18   SIGNAL op_or: STD_LOGIC_VECTOR(7 DOWNTO 0);
19   SIGNAL op_xor: STD_LOGIC_VECTOR(7 DOWNTO 0);
20   SIGNAL op_and: STD_LOGIC_VECTOR(7 DOWNTO 0);
21   SIGNAL op_addc: STD_LOGIC_VECTOR(7 DOWNTO 0);
22   SIGNAL op_add: STD_LOGIC_VECTOR(7 DOWNTO 0);
23   SIGNAL op_subc: STD_LOGIC_VECTOR(7 DOWNTO 0);
24   SIGNAL op_sub: STD_LOGIC_VECTOR(7 DOWNTO 0);
25   --sinais das operacoes logicas, de deslocamento e de rotacao
26   SIGNAL op_not: STD_LOGIC_VECTOR(7 DOWNTO 0);
27   SIGNAL op_sra: STD_LOGIC_VECTOR(7 DOWNTO 0);
28   SIGNAL op_srl: STD_LOGIC_VECTOR(7 DOWNTO 0);
29   SIGNAL op_sll: STD_LOGIC_VECTOR(7 DOWNTO 0);
30   SIGNAL op_rrc: STD_LOGIC_VECTOR(7 DOWNTO 0);
31   SIGNAL op_rlc: STD_LOGIC_VECTOR(7 DOWNTO 0);
32   SIGNAL op_rr: STD_LOGIC_VECTOR(7 DOWNTO 0);
33   SIGNAL op_rl: STD_LOGIC_VECTOR(7 DOWNTO 0);
34   --carry das operacoes de deslocamento e rotacao
35   SIGNAL carry_flag_sra, carry_flag_srl, carry_flag_sll: STD_LOGIC;
36   SIGNAL carry_flag_rrc, carry_flag_rlc, carry_flag_rr, carry_flag_rl:
       STD_LOGIC;
37   --flags
```

```
38   SIGNAL carry_flag, halfcarry_flag, overflow_flag: STD_LOGIC;
39   --auxiliares para a operacao de soma com carry
40   SIGNAL aux_somac_1: STD_LOGIC_VECTOR(8 DOWNTO 0);
41   SIGNAL aux_somac_2: STD_LOGIC_VECTOR(4 DOWNTO 0);
42   SIGNAL aux_carry_1: STD_LOGIC_VECTOR(8 DOWNTO 0);
43   SIGNAL aux_carry_2: STD_LOGIC_VECTOR(4 DOWNTO 0);
44   SIGNAL carry_flag_addc, halfcarry_flag_addc, overflow_flag_addc: STD_LOGIC;
45   --auxiliares para a operacao de soma sem carry
46   SIGNAL aux_soma_1: STD_LOGIC_VECTOR(8 DOWNTO 0);
47   SIGNAL aux_soma_2: STD_LOGIC_VECTOR(4 DOWNTO 0);
48   SIGNAL carry_flag_add, halfcarry_flag_add, overflow_flag_add: STD_LOGIC;
49   --auxiliares para a operacao de subtracao com carry
50   SIGNAL aux_subc_1: STD_LOGIC_VECTOR(8 DOWNTO 0);
51   SIGNAL aux_subc_2: STD_LOGIC_VECTOR(4 DOWNTO 0);
52   --como a operacao de add com carry ja tem auxiliares para o carry, podemos
        reaproveitar
53   SIGNAL carry_flag_subc, halfcarry_flag_subc, overflow_flag_subc: STD_LOGIC;
54   --auxiliares para a operacao de subtracao sem carry
55   SIGNAL aux_sub_1: STD_LOGIC_VECTOR(8 DOWNTO 0);
56   SIGNAL aux_sub_2: STD_LOGIC_VECTOR(4 DOWNTO 0);
57   SIGNAL carry_flag_sub, halfcarry_flag_sub, overflow_flag_sub: STD_LOGIC;
58
59 BEGIN
60
61   --primeira operacao 0000 entrada de B vai para a saida
62   pass_b <= entrada2;
63
64   --segunda operacao 0001 a saida e A or B
65   op_or <= entrada1 OR entrada2;
66
67   --terceira operacao 0010 a saida e A xor B
68   op_xor <= entrada1 XOR entrada2;
69
70   --quarta operacao 0011 a saida e A and B
71   op_and <= entrada1 AND entrada2;
72
73   --quinta operacao 0100 a saida e A+B+carry
74   --primeiramente precisamos estender o tamanho do sinal de carry para podermos
        somar
75   aux_carry_1 <= "00000000" & carry;
76   aux_carry_2 <= "0000" & carry;
77   -- operandos estendidos com '0' & para o carry nao se perder
78   aux_somac_1 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1)+UNSIGNED('0' &
        entrada2)+UNSIGNED(aux_carry_1));
79   aux_somac_2 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1(3 DOWNTO 0))+UNSIGNED
        ('0' & entrada2(3 DOWNTO 0))+UNSIGNED(aux_carry_2));
80   op_addc <= aux_somac_1(7 DOWNTO 0);
```

```
81  carry_flag_addc <= aux_somac_1(8);
82  halfcarry_flag_addc <= aux_somac_2(4);
83  overflow_flag_addc <= (entrada1(7) and entrada2(7) and not op_addc(7)) or (
        not entrada1(7) and not entrada2(7) and op_addc(7));
84  --fim soma com carry
85
86  --soma sem carry
87  aux_soma_1 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1)+UNSIGNED('0' &
        entrada2));
88  aux_soma_2 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1(3 DOWNTO 0))+UNSIGNED
        ('0' & entrada2(3 DOWNTO 0)));
89  op_add <= aux_soma_1(7 DOWNTO 0);
90  carry_flag_add <= aux_soma_1(8);
91  halfcarry_flag_add <= aux_soma_2(4);
92  overflow_flag_add <= (entrada1(7) and entrada2(7) and not op_add(7)) or (not
        entrada1(7) and not entrada2(7) and op_add(7));
93  --fim soma sem carry
94
95  --subtracao com carry
96  aux_subc_1 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1)-UNSIGNED('0' &
        entrada2)-UNSIGNED(aux_carry_1));
97  aux_subc_2 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1(3 DOWNTO 0))-UNSIGNED
        ('0' & entrada2(3 DOWNTO 0))-UNSIGNED(aux_carry_2));
98  op_subc <= aux_subc_1(7 DOWNTO 0);
99  carry_flag_subc <= aux_subc_1(8);
100 halfcarry_flag_subc <= aux_subc_2(4);
101 overflow_flag_subc <= (entrada1(7) and not entrada2(7) and not op_subc(7)) or
         (not entrada1(7) and entrada2(7) and op_subc(7));
102 --fim subtracao com carry
103
104 --subtracao sem carry
105 aux_sub_1 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1)-UNSIGNED('0' &
        entrada2));
106 aux_sub_2 <= STD_LOGIC_VECTOR(UNSIGNED('0' & entrada1(3 DOWNTO 0))-UNSIGNED
        ('0' & entrada2(3 DOWNTO 0)));
107 op_sub <= aux_sub_1(7 DOWNTO 0);
108 carry_flag_sub <= aux_sub_1(8);
109 halfcarry_flag_sub <= aux_sub_2(4);
110 overflow_flag_sub <= (entrada1(7) and not entrada2(7) and not op_sub(7)) or (
        not entrada1(7) and entrada2(7) and op_sub(7));
111 --fim subtracao sem carry
112
113 --nona operacao 1000 a saida e NOT A (carry = '0')
114 op_not <= NOT entrada1;
115
116 --decima operacao 1001 SRA: deslocamento aritmetico para a direita
117 --o bit 7 e repetido e o bit 0 vai para o carry
```

```
118   op_sra <= entrada1(7) & entrada1(7 DOWNTO 1);
119   carry_flag_sra <= entrada1(0);
120
121   --decima primeira operacao 1010 SRL: deslocamento logico para a direita
122   --entra '0' no bit 7 e o bit 0 vai para o carry
123   op_srl <= '0' & entrada1(7 DOWNTO 1);
124   carry_flag_srl <= entrada1(0);
125
126   --decima segunda operacao 1011 SLL: deslocamento logico para a esquerda
127   --entra '0' no bit 0 e o bit 7 vai para o carry
128   op_sll <= entrada1(6 DOWNTO 0) & '0';
129   carry_flag_sll <= entrada1(7);
130
131   --decima terceira operacao 1100 RRC: rotacao para a direita atraves do carry
132   --o carry de entrada vai para o bit 7 e o bit 0 vai para o carry de saida
133   op_rrc <= carry & entrada1(7 DOWNTO 1);
134   carry_flag_rrc <= entrada1(0);
135
136   --decima quarta operacao 1101 RLC: rotacao para a esquerda atraves do carry
137   --o carry de entrada vai para o bit 0 e o bit 7 vai para o carry de saida
138   op_rlc <= entrada1(6 DOWNTO 0) & carry;
139   carry_flag_rlc <= entrada1(7);
140
141   --decima quinta operacao 1110 RR: rotacao para a direita
142   --o bit 0 vai para o bit 7 e tambem para o carry
143   op_rr <= entrada1(0) & entrada1(7 DOWNTO 1);
144   carry_flag_rr <= entrada1(0);
145
146   --decima sexta operacao 1111 RL: rotacao para a esquerda
147   --o bit 7 vai para o bit 0 e tambem para o carry
148   op_rl <= entrada1(6 DOWNTO 0) & entrada1(7);
149   carry_flag_rl <= entrada1(7);
150
151   WITH operacao SELECT
152     resultado <= pass_b WHEN "0000",
153            op_or WHEN "0001",
154            op_xor WHEN "0010",
155            op_and WHEN "0011",
156            op_addc WHEN "0100",
157            op_add WHEN "0101",
158            op_subc WHEN "0110",
159            op_sub WHEN "0111",
160            op_not WHEN "1000",
161            op_sra WHEN "1001",
162            op_srl WHEN "1010",
163            op_sll WHEN "1011",
164            op_rrc WHEN "1100",
```

```
165             op_rlc WHEN "1101",
166             op_rr WHEN "1110",
167             op_rl WHEN "1111",
168             "00000000" WHEN OTHERS;
169
170   --nas operacoes logicas (0000 a 0011 e 1000) o carry e '0'
171   WITH operacao SELECT
172     carry_flag <= carry_flag_addc WHEN "0100",
173             carry_flag_add WHEN "0101",
174             carry_flag_subc WHEN "0110",
175             carry_flag_sub WHEN "0111",
176             carry_flag_sra WHEN "1001",
177             carry_flag_srl WHEN "1010",
178             carry_flag_sll WHEN "1011",
179             carry_flag_rrc WHEN "1100",
180             carry_flag_rlc WHEN "1101",
181             carry_flag_rr WHEN "1110",
182             carry_flag_rl WHEN "1111",
183             '0' WHEN OTHERS;
184
185   --halfcarry: so tem significado nas operacoes aritmeticas (nas demais e don't
        care, usamos '0')
186   WITH operacao SELECT
187     halfcarry_flag <= halfcarry_flag_addc WHEN "0100",
188               halfcarry_flag_add WHEN "0101",
189               halfcarry_flag_subc WHEN "0110",
190               halfcarry_flag_sub WHEN "0111",
191               '0' WHEN OTHERS;
192
193   --overflow: so tem significado nas operacoes aritmeticas (nas demais e don't
       care, usamos '0')
194   WITH operacao SELECT
195     overflow_flag <= overflow_flag_addc WHEN "0100",
196               overflow_flag_add WHEN "0101",
197               overflow_flag_subc WHEN "0110",
198               overflow_flag_sub WHEN "0111",
199               '0' WHEN OTHERS;
200
201   saida <= resultado;
202
203   flags(0) <= resultado(7);                              -- negative
204   flags(1) <= overflow_flag;                             -- overflow
205   flags(2) <= '1' WHEN resultado = "00000000" ELSE '0'; -- zero
206   flags(3) <= halfcarry_flag;                            -- half carry / borrow
207   flags(4) <= carry_flag;                                -- carry / borrow
208
209 END ula;
```